



Svelte Review Cards

Eight concepts an interviewer can probe and a beginner must own. Read the question, cover the answer, answer aloud, then check the code and the figure.

The Loop Danger

Q26

REACTIVITY

INTERVIEW QUESTION

Question. How does reactivity work traditionally, and what is the danger when an effect modifies a state it depends on?

ANSWER. In Svelte 3 and 4, reactivity is achieved by looking for variable assignments. At compile time, when the compiler sees an assignment like `count += 1`, it invisibly wraps that assignment in an invalidate function that immediately tells the precise associated DOM node to update directly.

App.svelte

```
<script>
  // Svelte 4 Reactivity
  let count = 0;

  function add() {
    count += 1; // Triggers UI update
  }
</script>

<button on:click={add}>
  Clicks: {count}
</button>
```

`count = 1`



COMPILER



`$$invalidate('count', ...)`

DOM UPDATES DIRECTLY

- ① **Summary.** Reactivity in older Svelte versions relies on explicit variable assignments. The compiler statically analyzes the code and injects direct DOM updates wherever it detects these assignments, completely bypassing the need for a Virtual DOM.

The Assignment Operator in Svelte 3/4

Q27

STATE (LEGACY)

INTERVIEW QUESTION

Question. What is the operational significance of the assignment operator (=) in Svelte 3/4 reactivity?

ANSWER. The assignment operator (=) is the fundamental trigger for reactivity. The compiler detects assignments to top-level variables and explicitly injects an internal `$$invalidate` function call to queue a DOM update. Mutating data via internal methods like `push()` without reassignment bypasses this detection.

App.svelte

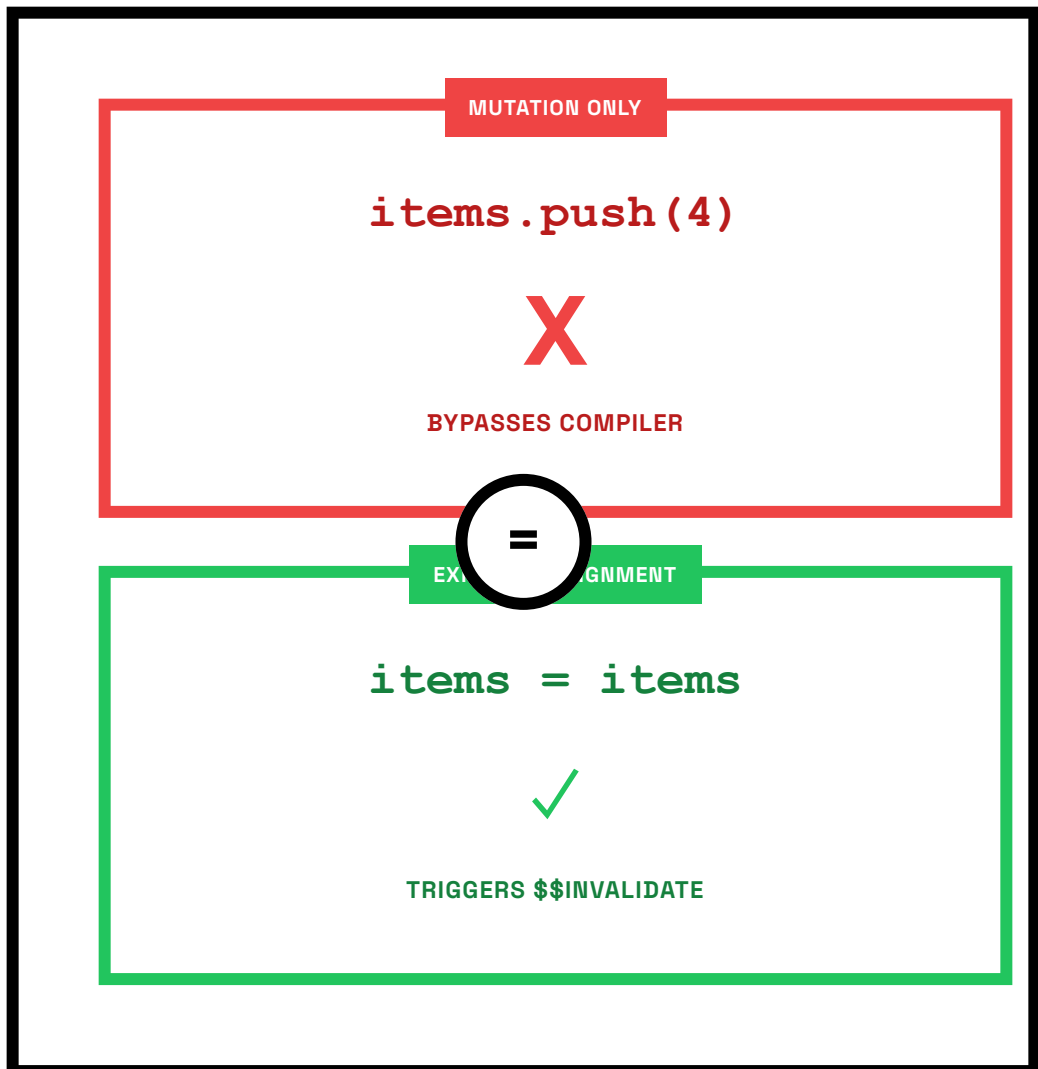
```
<script>
  let items = [1, 2, 3];

  function addItem() {
    // A method mutation alone does NOT trigger reactivity
    items.push(4);

    // The assignment operator (=) signals the compiler to
    update the DOM
    items = items;
  }
</script>

<ul>
  {#each items as item}
    <li>{item}</li>
```

```
{/each}  
</ul>
```



① **Summary.** Mutating an array or object in Svelte 3 and 4 is not enough. You must follow any mutation with an explicit assignment to trigger the compiler's reactivity tracking and update the DOM.

Array Mutations in Legacy Svelte

Q28

STATE (LEGACY)

INTERVIEW QUESTION

Question. Why does mutating an array using methods like `.push()` fail to trigger a reactive update by default in legacy Svelte?

ANSWER. In Svelte 3 and 4, reactivity is driven strictly by variable assignment (`=`). Array methods like `.push()`, `.pop()`, or `.splice()` mutate the underlying array in memory without reassigning the variable reference itself. Because the Svelte compiler instruments assignments rather than observing memory mutations at runtime, the invalidation routine is never triggered unless the variable is explicitly reassigned.

App.svelte

```
<script>
  let items = ['Apple', 'Banana'];

  function addItem() {
    // ❌ Mutates array in-place, but does NOT reassign 'items'
    items.push('Cherry');

    // ✅ Legacy fix: Reassignment triggers Svelte's compile-
    // time invalidation
    items = items;
  }
</script>

<button on:click={addItem}>Add Item</button>
```

```
<ul>
  {#each items as item}
    <li>{item}</li>
  {/each}
</ul>
```

- ❗ **Summary.** The Svelte 3/4 compiler cannot track internal mutations to objects or arrays at runtime. You must always use the assignment operator to notify Svelte that a variable's state has changed and the DOM needs an update.

NO REASSIGNMENT (UI IGNORED)

`items.push('Cherry')`

X

ASSIGNMENT DETECTED (UI UPDATED)

`items = items`

✓

Object Property Mutations in Legacy Svelte

Q29

STATE (LEGACY)

INTERVIEW QUESTION

Question. How do you force a reactive update after modifying an object property in Svelte 3/4?

ANSWER. In Svelte 3 and 4, modifying a nested property on an object (e.g., `user.age = 31`) does not automatically trigger the reactivity system. Because the compiler tracks assignments at the top-level identifier to inject invalidation code, you must explicitly reassign the object to itself (`user = user`) or assign it a completely new object reference (often using the spread operator) to force the UI to render the updated state.

App.svelte

```
<script>
  let user = { name: 'Alice', age: 30 };

  function updateAge() {
    // ❌ Mutating a nested property does NOT trigger an update
    user.age = 31;

    // ✅ Legacy fix 1: Reassign the object to itself
    user = user;

    // ✅ Legacy fix 2: Reassign using the spread operator
    // user = { ...user, age: 31 };
  }
}
```



```
</script>
```

```
<button on:click={updateAge}>  
  {user.name} is {user.age} years old  
</button>
```

- ① **Summary.** Mutating an object property directly avoids detection by the compiler in Svelte 3 and 4. You must always use an explicit top-level assignment to force the UI to react to the updated state.

PROPERTY MUTATION (SILENT)

`user.age = 31`

TOP-LEVEL ASSIGNMENT (REACTIVE)

`user = user`

Reactive Statements

Q30

STATE (LEGACY)

INTERVIEW QUESTION

Question. What is the purpose and syntactic structure of the `$:` labeled statement in Svelte 3/4?

ANSWER. In Svelte 3 and 4, the `$:` prefix (which utilizes standard JavaScript label syntax) is used to construct reactive statements and reactive declarations. The compiler automatically analyzes the code following the label, infers the local variables acting as dependencies, and instruments the compiled output to re-execute that specific block of code whenever those dependencies change. A reactive declaration defines a new variable (e.g., `$: x = y * 2`), while a reactive statement runs arbitrary side-effects without variable declaration (e.g., `$: console.log(y)`).

App.svelte

```
<script>
  let count = 0;

  // Reactive declaration: creates a computed variable
  'double'
  $: double = count * 2;

  // Reactive statement: triggers arbitrary side-effects
  $: console.log('Count updated:', count);

  // Reactive block: executes multiple lines upon dependency
  change
  $: {
```

```
if (count > 10) {  
  alert('Threshold reached');  
}  
}  
</script>
```

- ① **Summary.** The `$:` label marks code that should automatically re-run. Svelte handles the dependency tracking statically during compilation, meaning any referenced local reactive variables will automatically trigger the statement when updated.



MAGIC LABEL

**Compiler auto-
tracks
dependencies
here**

Compiler Dependency Inference

Q31

STATE (LEGACY) COMPILER

INTERVIEW QUESTION

Question. How does the Svelte compiler automatically infer dependencies for a legacy `$:` reactive statement?

ANSWER. In legacy Svelte (v3/v4), the compiler acts as a static analysis tool. When it encounters the `$:` label, it parses the trailing expression into an Abstract Syntax Tree (AST). It scans this tree to locate any top-level component variables that are read or evaluated within the statement. Svelte then automatically instruments the compiled JavaScript so that whenever any of those identified variables are reassigned, the reactive statement is queued to re-run.

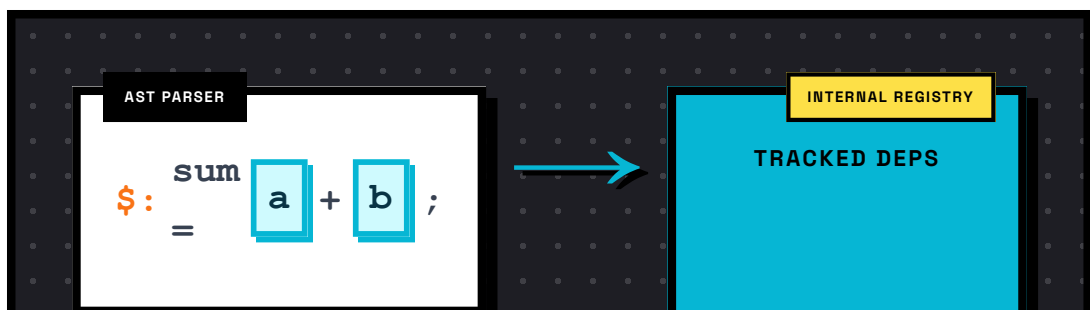
App.svelte

```
<script>
  let a = 1;
  let b = 2;
  let c = 3;

  // The compiler sees 'a' and 'b' being read.
  // It registers them as dependencies for this block.
  // 'c' is ignored because it is not evaluated here.
  $: sum = a + b;

  // For functions, the compiler only looks at the arguments passed or variables
  // explicitly read inside the `$:` line itself, NOT inside the function body!
  $: result = calculate(a); // Dependency: 'a' and 'calculate', but NOT variables inside
  'calculate'
</script>
```

① **Summary.** The legacy Svelte compiler relies on strict lexical analysis. If a top-level variable is visibly read inside the `$:` statement, it becomes a dependency. Variables hidden inside external function calls are completely invisible to the dependency tracker.





The diagram shows a large dark gray rectangle with a grid of small dots, representing a 2D array. A smaller, lighter gray rectangle is positioned in the top right corner of the large rectangle. Inside this lighter rectangle, a white square is outlined, and within it, a cyan square is outlined. The text "[a, b]" is written in white inside the white square. The entire diagram is framed by a thick black border on the left and right sides.

```
[ a, b ]
```

Reactive Blocks

Q32

STATE (LEGACY) BLOCK LOGIC

INTERVIEW QUESTION

Question. Can a legacy `$:` statement block execute multi-line logic, blocks, or external function calls? Provide a conceptual example.

ANSWER. Yes. Because `$:` is simply valid JavaScript syntax (a labeled statement), it can precede any statement. By combining it with a curly brace block `{ ... }`, you can execute multi-line logic, conditional `if/else` statements, or trigger external function calls. The compiler will scan the entire block and run it whenever any of its internal top-level dependencies are updated.

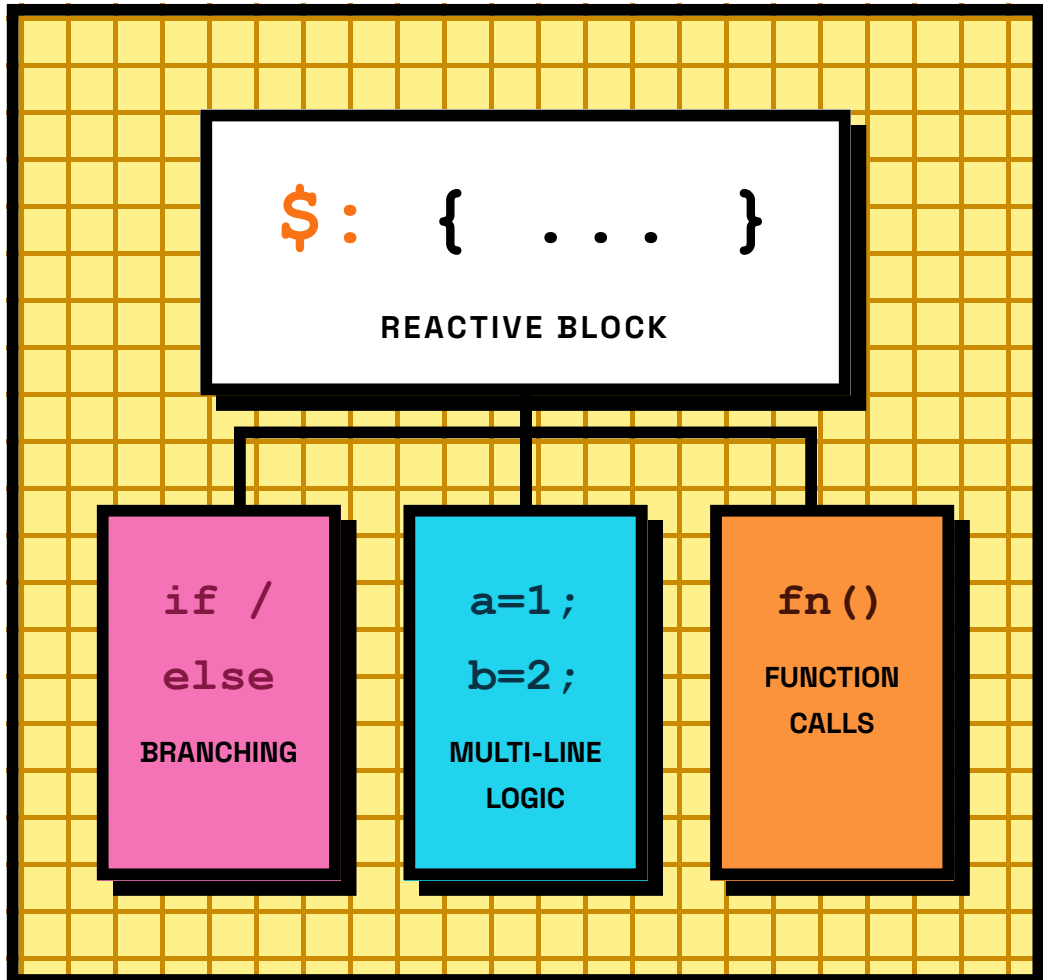
App.svelte

```
<script>
  let score = 85;
  let grade = 'F';

  // The $: label applies to the entire block.
  // When 'score' changes, the entire block runs again.
  $: {
    if (score ≥ 90) {
      grade = 'A';
      triggerConfetti(); // External function
    } else if (score ≥ 80) {
      grade = 'B';
    } else {
      grade = 'F';
      logFailure(score);
    }
  }
}
```

</script>

- ① **Summary.** By attaching the `$:` label to a block `{ ... }`, you can wrap complex, multi-line conditional logic and side-effects into a single reactive bundle that re-evaluates automatically based on its dependencies.



INTERVIEW QUESTION

Question. What are Svelte 5 Runes, and what specific architectural limitations of legacy reactivity do they address?

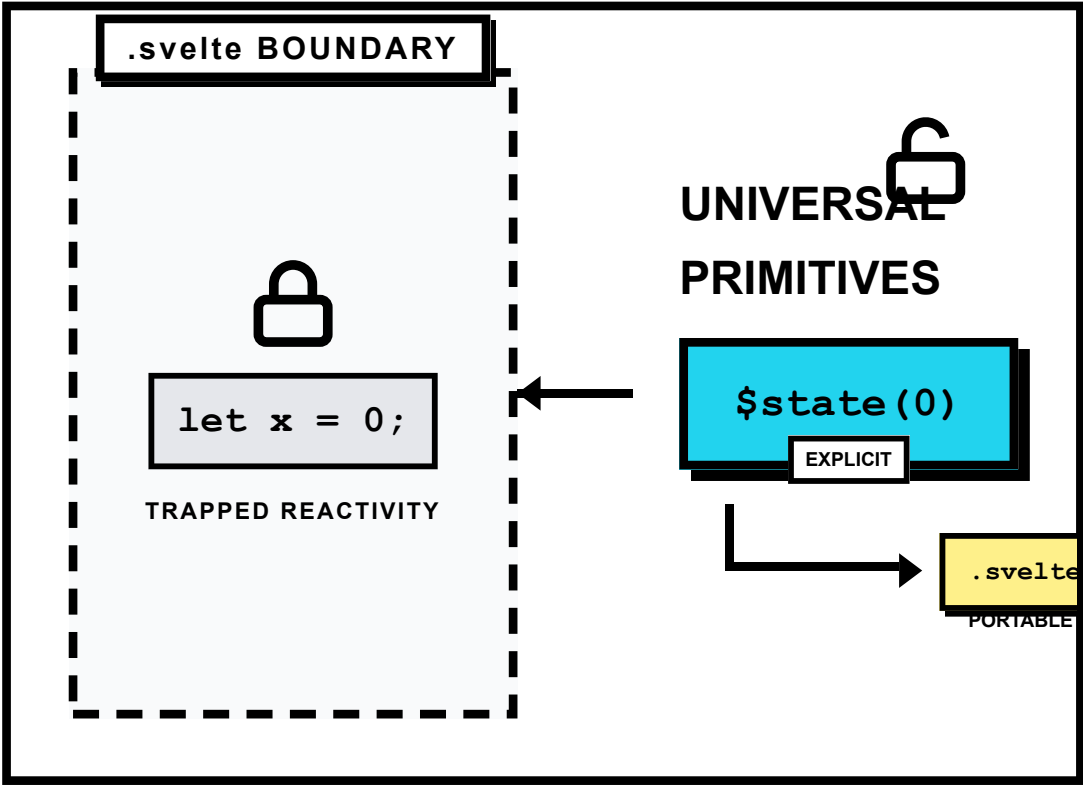
ANSWER. Svelte 5 Runes are explicitly declared, fine-grained reactive primitives (such as `$state`, `$derived`, and `$effect`). They replace the legacy component-level reactivity system to address three major architectural limitations: 1. **UNIVERSAL STATE:** Legacy reactivity was strictly bound to `.svelte` files. Runes can be declared in raw `.svelte.js` or `.ts` files, making it effortless to share state across the entire application without using stores. 2. **EXPLICIT DEEP REACTIVITY:** Legacy Svelte relied on the assignment operator (`=`) to trigger updates, which caused confusion when mutating nested arrays or objects. Runes use an explicit Proxy-based signal system, so deep object mutations trigger updates automatically. 3. **FINE-GRAINED DOM UPDATES:** Legacy reactivity dirty-checked the entire component on every state change. Runes use a signal graph to pinpoint exact DOM nodes that depend on the state, bypassing component-level checks and vastly improving performance.

shared.svelte.js

```
// With Runes, reactive state can finally live outside components!
export function createCounter() {
  let count = $state(0);

  return {
    get count() { return count; },
    increment() { count += 1; }
  };
}
```

i Summary. Runes untangle reactivity from the `.svelte` component boundary. They offer explicit, deeply reactive, and surgically fine-grained state management that scales gracefully into large codebases.



INTERVIEW QUESTION

Question. Explain the core purpose, syntax, and execution model of the `$state` rune.

ANSWER. The `$state` rune is the fundamental building block of reactivity in Svelte 5.

1. **Core Purpose:** It declares a variable as reactive. Unlike standard JavaScript variables, changing a `$state` variable guarantees that any part of the UI or derived logic depending on it will automatically update. 2. **Syntax:** It is invoked as a function wrapping the initial value: `let count = $state(0);`. 3. **Execution Model:** Under the hood, `$state` creates a reactive "signal." When the variable is read during a render, Svelte records the DOM node as a dependency. When the variable is updated, `$state` acts like a broadcaster, sending a direct, fine-grained signal to update *only* those specific DOM nodes, bypassing full component re-renders entirely.

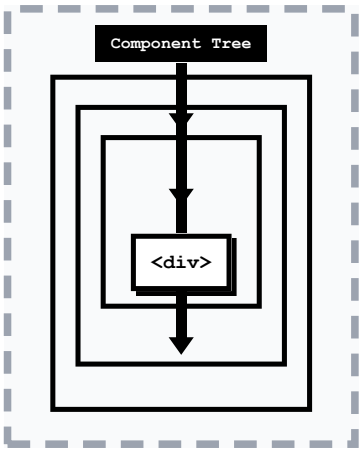
Broadcaster.svelte

```
<script>
  // Declaring a live signal
  let message = $state('Hello!');

  function update() {
    // Re-assigning triggers the broadcast
    message = 'Runes are active.';
  }
</script>

<!-- This specific node listens to the 'message' signal -->
<h1>{message}</h1>
<button onclick={update}>Update</button>
```

LEGACY: TOP-DOWN



RUNES: DIRECT UPDATE



\$state Deep Reactivity

Q35

SVELTE 5 RUNES DEEP REACTIVITY STATE MANAGEMENT

INTERVIEW QUESTION

Question. How does the `$state` rune handle deep reactivity within nested objects and arrays?

ANSWER. Unlike legacy Svelte which relied on explicit reassignments to trigger reactivity on objects and arrays, the `$state` rune is fundamentally deeply reactive by default. When you pass an object or array to `$state(value)`, Svelte automatically wraps it (and all of its deeply nested properties) in a reactive Proxy.

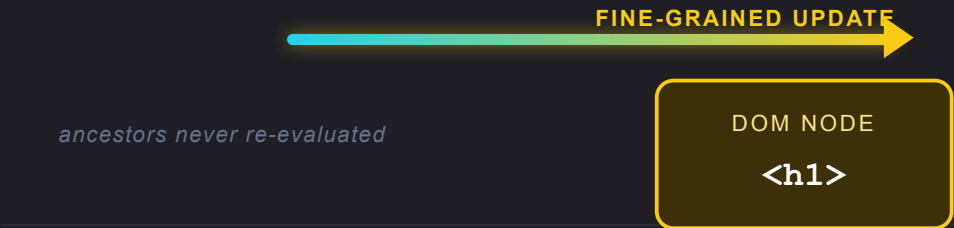
This means that you can mutate a deeply nested property directly, or push to an array, and the fine-grained signal system will perfectly detect the granular change and update only the targeted DOM nodes that depend on that specific leaf node.

```
<script>
  let user = $state({
    profile: {
      name: "Alice",
      preferences: { theme: "dark" }
    }
  });

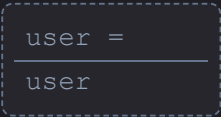
  // This deeply nested mutation triggers a precise update
  automatically
  function toggleTheme() {
    user.profile.preferences.theme =
      user.profile.preferences.theme === "dark" ? "light" : "dark";
  }
</script>
```

```
<button onClick={toggleTheme}>
  Theme: {user.profile.preferences.theme}
</button>
```

\$STATE DEEP PROXY WRAPPED ON ACCESS, NOT ON CREATION



LEGACY RITUAL, NOW OBSOLETE



BULK INVALIDATE

`$state.frozen` / `$state.raw`: Opting Out of Deep Reactivity

Q36

SVELTE 5 RUNES `$STATE.RAW` PERFORMANCE

INTERVIEW QUESTION

Question. What is the `$state.frozen` rune, and in what scenarios should it be chosen over standard `$state`?

ANSWER. The rune that was prototyped as `$state.frozen` shipped in Svelte 5 as `**$state.raw*`. *It declares reactive state that Svelte does not* wrap in a deep Proxy: assignment to the variable itself is reactive (`config = newConfig` triggers updates), but mutating a nested property (`config.theme = 'dark'`) is silent — there is no proxy to detect it, so no update fires.

Use it whenever you hold **read-mostly structured data that you replace wholesale instead of mutate in place**: configuration objects, look-up tables, parsed fixtures, feature flags, immutable shapes received from a store. The payoff is concrete: Svelte skips the per-property proxy allocation on every access, so cold reads and initial mount of large objects are cheaper, and you stop paying for fine-grained tracking you never use.

The trap is symmetric: reaching for `$state.raw` to “optimize” a value you then mutate at depth is the same bug as legacy Svelte’s `user.age = 31` — the change lands in memory but the UI never hears about it. The rule is prescriptive, not stylistic: deep reactivity is the default because it is always correct; `$state.raw` is an opt-out you take only when you commit to the replace-don’t-mutate discipline for that value.

```
<script>
  // Deeply reactive: every nested property is tracked.
  // Use when you will mutate fields in place.
```

App.svelte

```

let session = $state({ user: { name: 'Ada', role: 'admin' } });

// Not deeply reactive: assignment is reactive, mutation is not.
// Use when you always replace the whole value.
let config = $state.raw({
  apiUrl: 'https://api.example.com',
  retries: 3,
  features: { beta: false },
});

function patchConfig(next) {
  // ✅ Replace the reference → UI updates.
  config = { ...config, ...next };
}

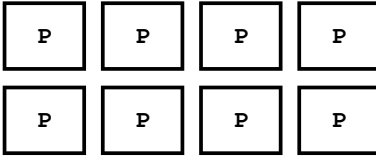
function misuseConfig() {
  // ❌ Silent: no proxy here, so this change never reaches the
  DOM.
  config.features.beta = true;
}
</script>

<p>{session.user.name} ({session.user.role})</p>
<p>API: {config.apiUrl}, retries: {config.retries}</p>
<button onClick={() => patchConfig({ retries: 5 })}>Bump
retries</button>

```

❶ **Summary.** `$state.raw` (the rune prototyped as `$state.frozen`) is the deliberate off-switch for deep proxying. Choose it for read-mostly structured data that you replace rather than mutate, and leave `$state` as the default everywhere else — because only the deep variant is correct when you need in-place edits to render.

`$STATE (HUGEARRAY)`



PROXY ON EVERY PROPERTY

`$STATE .RAW (HUGEARRAY)`

A diagram consisting of a single yellow rectangular box with a black border. Inside the box, the text `[ ...one value... ]` is written in a monospace font.

`[...one
value...]`

TRACKED ONLY AT THE
BINDING

\$derived Rune

Q37

SVELTE 5 RUNES REACTIVITY OPTIMIZATION

ANSWER. The `$derived` rune calculates derived state synchronously from other state variables. When any of the source `$state` signals it depends on change, the `$derived` value automatically re-evaluates.

Its primary optimization benefit is memoization. If the underlying state hasn't changed, Svelte returns the cached value instead of re-running the computation. This prevents expensive recalculations and unnecessary DOM updates. Additionally, `$derived` tracks its dependencies dynamically at runtime, avoiding stale closures.

App.svelte

```
<script>
  let numbers = $state([1, 2, 3, 4, 5]);

  // Automatically recalculates only when 'numbers' changes
  let total = $derived(numbers.reduce((a, b) => a + b, 0));
</script>

<div>The total is {total}</div>
```

❶ **Summary.** Use `$derived` for computing values derived from `$state`. It offers automatic dependency tracking and memoization, recalculating only when necessary to optimize performance.

Input 1

`$state(A)`

Input 2

`$state(B)`

Computation
\$derived(A + B)
Memoized Evaluation
Output
Cached Result

\$derived.by: The Block-Body Form

Q38

SVELTE 5 RUNES \$DERIVED.BY COMPUTED VALUES

INTERVIEW QUESTION

Question. How does the `$derived.by` rune differ from a standard `$derived` expression, and when is it necessary?

ANSWER. `$derived` and `$derived.by` are the same mechanism wearing two syntactic shapes. The plain form `$derived(expr)` takes a single expression; `.by` takes a zero-argument function whose body is a block of statements, and returns whatever the block returns. Under the hood they compile to the same memoized, lazily-recomputed, dependency-tracked signal — the dependency tracking, caching, and laziness rules are identical to Q37.

`.by` is necessary in exactly one situation: **when the computation cannot be written as a single expression**. The rule is prescriptive, not stylistic — use `$derived(expr)` whenever you can, and reach for `$derived.by(() => { ... })` only when you genuinely need statements. The four common triggers are: a `let` for an intermediate variable, a `for` or `for...of` loop that accumulates a result, a `try/catch` around something that may throw, or several sequential calculations that read more clearly as steps than as one tangled ternary. If you find yourself writing `$derived.by(() => { return expr; })` — a function whose body is just a single return — you have written the wrong form; collapse it back to `$derived(expr)`.

```
<script>
  let items = $state([10, 20, 30]);
  let tax = $state(0.2);

  // Single expression → plain $derived.
  let count = $derived(items.length);

  // Needs statements (a loop + an intermediate let) →
  $derived.by.
  let totalWithTax = $derived.by(() => {
    let subtotal = 0;
    for (const item of items) subtotal += item;
    return subtotal * (1 + tax);
  });
</script>

<p>{count} items, total: {totalWithTax}</p>
```

❶ **Summary.** `$derived(expr)` and `$derived.by(() => { ... })` are the same memoized signal under the hood — `.by` exists only for computations that need statements (loops, intermediate `let`s, `try/catch`, multi-step logic). Reach for plain `$derived` first; switch to `.by` only when an expression genuinely cannot carry the work.

SAME MEMOIZED SIGNAL UNDERNEATH

`$derived(expr)`

one expression

REACH FOR THIS FIRST

`$derived.by(() => {
... })`

`let subtotal = 0;`

`for (...) subtotal
+= ...;`

`return subtotal *
(1 + tax);`

STATEMENTS ALLOWED

\$inspect Rune

Q39

SVELTE 5 RUNES DEBUGGING STATE MANAGEMENT

ANSWER. The `$inspect` rune is a powerful built-in debugging utility that acts like a reactive `console.log`. Instead of manually logging state changes inside an `$effect`, you simply pass a reactive signal (like `$state` or `$derived`) to `$inspect()`.

Whenever the tracked state changes, Svelte automatically logs the new value to the browser console. It strips away the Proxy overhead, allowing you to clearly see the raw underlying data structure without the noise of the reactivity system. It is strictly a development tool and is stripped out entirely during production builds.

Debug.svelte

```
<script>
  let count = $state(0);
  let double = $derived(count * 2);

  // Automatically logs to the console whenever count or
  double changes
  $inspect(count, double);
</script>

<button onclick={() => count++}>Increment</button>
```

① **Summary.** Use the `$inspect` rune to cleanly log reactive state changes to the console. It automatically unwraps deep proxies for readability and is stripped out in production.

INPUT

```
Proxy({  
  val: Proxy(...)  
})
```



UNWRAP

```
Q $inspect()
```



CONSOLE

```
{ val: 42 }
```

Microtask Batching

Q46

SVELTE 5 REACTIVITY MICROTASK PERFORMANCE

INTERVIEW QUESTION

Question. What is the microtask batching mechanism in Svelte, and how does it protect against redundant UI renders?

ANSWER.

The mechanism — what happens on every write

- **Two-step write:** every `** $state` assignment** (a line like `count = 5`) does two things in sequence — it stores the new value **immediately**, and it tells Svelte to schedule **one flush** later. A **flush** is Svelte's term for the moment it actually updates the screen.
- **Scheduled, not immediate:** that flush is queued on the **microtask queue** — a built-in JavaScript list of tiny tasks that the browser runs *after* your current function finishes, but *before* it paints anything. So if you write to the same variable ten times in a row, all ten writes land **before the flush ever runs**.
- **What the flush does:** Svelte looks at every `** $derived` value (a **computed value that depends on others**) whose inputs changed, **recomputes only those, and then writes the final values**** to the DOM. Intermediate steps are skipped entirely.

The protection — why renders are not redundant

- **One render per tick:** a loop calling `count += 1` **ten times** produces **one DOM update** at the end, not ten. The flush sees the final value of `count` and acts on that alone.
- **Multiple variables coalesce:** if a handler updates **several variables at**

once, each is **flagged as changed** (Svelte marks "this one's stale, re-check it"), but Svelte processes **all of those flags in a single flush** that runs once after the handler returns — never one flush per variable.

- **No wasted renders:** Svelte **never paints** an intermediate value that the user would never see, because the screen only updates after the flush.

The rule — what you should and should not do

- **Do not debounce:** never reach for **`**setTimeout`** or manual batching **to combine reactive writes** — the runtime already does it**, so adding your own delay just makes the UI feel laggy.
- ****Do await `tick()` when you need to read the DOM** after it has settled** (see Q47). `tick()` is Svelte's function that returns a Promise resolving **once the current flush finishes** — the **only** legitimate reason to wait for a flush.

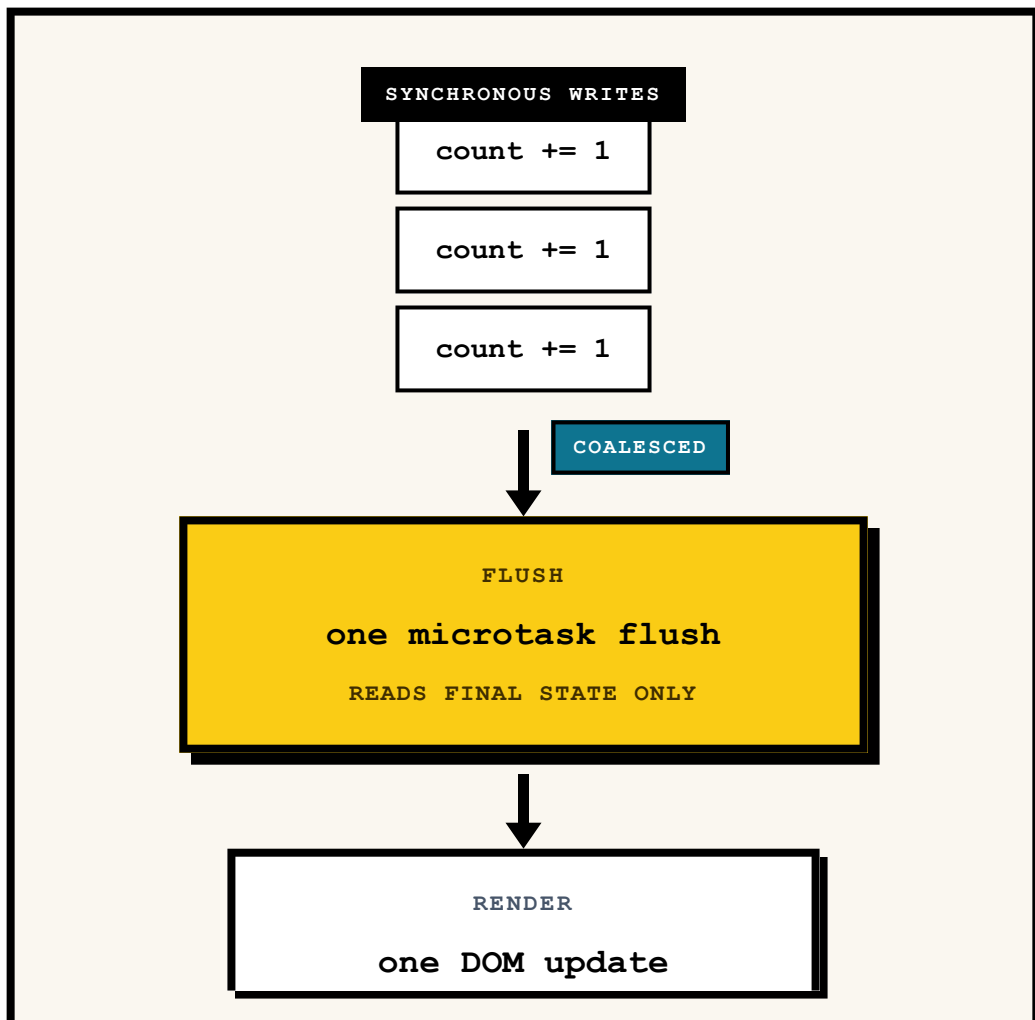
App.svelte

```
<script>
  let count = $state(0);
  let label = $state('ready');

  function burst() {
    // Ten synchronous writes — but Svelte schedules ONE
    flush.
    for (let i = 0; i < 10; i++) count += 1;
    label = `done at ${count}`;
    // When the microtask runs, count is 10 and label is 'done
    at 10'.
    // The DOM is updated exactly once with those final
    values.
  }
</script>

<button onclick={burst}>burst</button>
<p>{label}</p>
```


① **Summary.** Svelte schedules exactly one flush per microtask. Multiple synchronous writes to one or more signals coalesce into a single render of the final state, so intermediate values the user would never see are never written to the DOM. The runtime already batches — do not add manual debouncing on top.



count = 3

The `tick()` Function

Q47

SVELTE 5 REACTIVITY TICK DOM

INTERVIEW QUESTION

Question. How do you manually tap into the rendering cycle using Svelte's `tick` function, and what does it return?

ANSWER.

****What `tick()` is and what it returns****

- **Returns a Promise:** `tick()` is a function imported from `'svelte'` that returns a **Promise** — a built-in JavaScript object representing a value that will be available later. You `await` it inside an `async` function.
- **Resolves after the flush:** the Promise resolves (becomes ready) **once the current reactive flush finishes** — the flush being Svelte's term for the moment it walks the dirty signals and writes the final values to the DOM (see Q46).
- **No value carried:** `tick()` does **not return a value** when awaited. It is a synchronization primitive — its job is to give you a moment in time, not data.

Why you need it

- **DOM reads are stale mid-handler:** when you write `el.scrollTop = 0` right after a state change, the DOM has **not updated yet** because the flush is scheduled for after your handler returns. Reading `el.scrollHeight` at that instant returns the **old value**.
- **** `await tick()` unblocks the flush: by awaiting the Promise, you yield control back to JavaScript, the microtask queue runs, the flush**

executes and paints**, and execution resumes with the DOM in its new state.

- **The fix is one line:** `count += 1; await tick(); console.log(el.scrollHeight);` — now the log shows the **post-update measurement**.

The rules for correct use

- ****Only inside `async` functions:**** `await` is a JavaScript keyword that only works inside a function declared `async`. Using `await tick()` in a synchronous handler is a syntax error.
- **Use sparingly:** reach for `tick()` only when you genuinely need to **read or measure the DOM** after a state change. Most reactive code never needs it — Svelte's template bindings already reflect new state automatically.
- **Never use it to order side effects:** if you need code to run after state settles, prefer ****`$effect`**** (see Q20) over manual `tick(). tick()` answers *measurement*, `$effect` answers *reaction*.

App.svelte

```
<script>
  import { tick } from 'svelte';

  let items = $state(['a', 'b']);
  let listEl;

  async function addAndScroll() {
    items.push('c');           // State changed, but DOM is
    not updated yet.
    await tick();              // Wait for the flush to write
    the new <li>.
    listEl.scrollTop = listEl.scrollHeight; // Now measures
    the full list.
  }
</script>

<ul bind:this={listEl}>
  {#each items as item}<li>{item}</li>{/each}
</ul>
```

```
<button onclick={addAndScroll}>add and scroll</button>
```

① **Summary.** `tick()` is imported from `'svelte'` and returns a Promise that resolves after the current reactive flush finishes. `await` it inside an `async` function when you need to read or measure the DOM after a state change; for everything else, prefer `$effect`.

STATE

```
items.push('c')
```

STATE CHANGED, DOM STALE

YIELDS CONTROL

YIELD POINT

```
await tick()
```

HANDLER PAUSES, MICROTASK RUNS

FLUSH + RESUME

DOM is current

EXECUTION RESUMES, MEASUREMENTS VALID

Practical `tick()` Scenario: Auto-Scrolling a Chat Log

Q48

SVELTE 5 TICK DOM SCROLL PRACTICAL

INTERVIEW QUESTION

Question. Provide a practical scenario where awaiting `tick()` is required to read accurate DOM states after a state change.

ANSWER.

The scenario — a chat log that must auto-scroll to the bottom

- **The feature:** a chat component renders an array of messages with `{#each}`, and every time a new message arrives, the container must scroll to the **bottom** so the latest message is visible.
- **The measurement:** to scroll to the bottom you read `** el.scrollHeight (the full height of the content) and assign it to el.scrollTop (the scroll position). This is a DOM read followed by a DOM write**` that depends on the read.
- **The trap:** calling `messages.push(newMsg)` dirties state, but the new `<li>` is **not in the DOM yet** because the flush is scheduled for after the handler returns (see Q46). Reading `scrollHeight` at that instant returns the **pre-push height** — the new message is invisible, and the scroll lands one message short.

`**The fix — await tick() between the write and the read**`

- **Yield before measuring:** insert `** await tick() **` between the state change and the DOM read. `tick()` returns a **Promise** that resolves once the current **reactive flush** finishes — the flush being Svelte's term for the moment the new `<li>` is actually written to the DOM.

- **Now the read is accurate:** by the time execution resumes, the DOM reflects the new array, so `el.scrollHeight` includes the freshly pushed message, and `el.scrollTop = el.scrollHeight` lands exactly at the bottom.
- **The handler must be `async` and `await`** only works inside a function declared `async`, so the click/send handler must be `async function send()` rather than a plain function.

Why you cannot fake it with `setTimeout`

- `setTimeout(fn, 0)` is wrong: it schedules the callback on the macrotask queue, which runs after rendering — meaning the user sees one frame of the old scroll position before it jumps. `tick()` resolves on the microtask queue, before paint, so the scroll updates in the same frame as the new message.
- **No debounce either:** wrapping the scroll in a timeout adds visible latency that feels broken. `tick()` adds none — it waits only for the flush that was already going to happen.

Chat.svelte

```
<script>
  import { tick } from 'svelte';

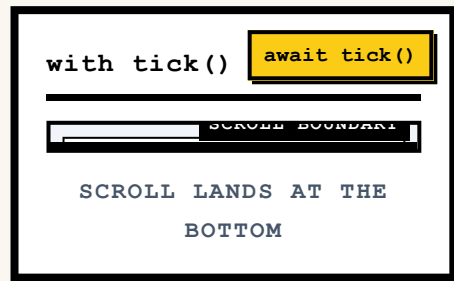
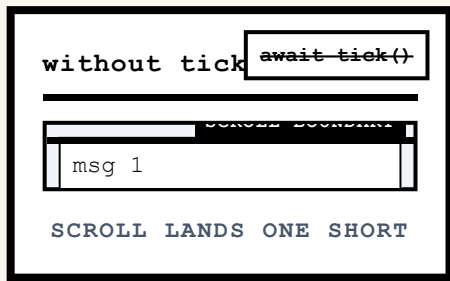
  let messages = $state(['hello', 'world']);
  let logEl;

  async function send() {
    messages.push('new message'); // State changed; <li> not
    in DOM yet.
    await tick(); // Wait for the flush to
    write the new <li>.
    logEl.scrollTop = logEl.scrollHeight; // Now scrollHeight
    includes the new row.
  }
</script>

<ul bind:this={logEl} style="height:120px; overflow:auto;">
  {#each messages as m}<li>{m}</li>{/each}
</ul>
```

```
<button onclick={send}>send</button>
```

① **Summary.** Auto-scrolling a chat log after `messages.push(...)` requires reading `scrollHeight`, but the new `<li>` is not in the DOM until the flush runs. `await tick()` between the push and the read guarantees the measurement reflects the new state — and unlike `setTimeout`, it resolves before paint, so there is no visible latency.



Syncing Local State with a Prop

Q49

SVELTE 5 RUNES \$STATE \$DERIVED PROPS

INTERVIEW QUESTION

Question. How do you initialize a reactive local variable that dynamically syncs with an incoming prop using Runes?

ANSWER.

The two patterns — pick by whether the local copy must be editable

- ****Editable local copy** → `$state(prop)` ****** declare `let local = $state(prop.value)` to create a **deeply reactive local variable** (see Q34) whose initial value is copied from the prop. The user can then mutate `local` freely — typing into a bound input, toggling, etc.
- ****Read-only mirror** → `$derived(prop.value)` ****** declare `let mirror = $derived(prop.value)` to create a **memoized computed value** (see Q37) that re-evaluates whenever the prop changes. You cannot assign to a `$derived` — it is a pure function of its inputs.

****Why `$state(prop)` alone does not sync****

- **One-shot initialization:** writing `let local = $state(prop.value)` copies the prop's value **once**, at component setup. The `$state(...)` call runs a single time, so later changes to the prop do **not** flow into `local`.
- **The prop is a moving target:** props in Svelte 5 are reactive — the parent can pass a new value at any time, and the child sees the update — but `$state(initialValue)` only reads `initialValue` at birth.

- **The fix when you need both:** to get an editable local copy that *also* re-syncs when the prop changes, combine the two — declare `let local = $state(prop.value)` and add a `$effect(() => { local = prop.value; })` that watches the prop and overwrites the local copy on change (with care; see Q21 about avoiding effect loops).

The prescriptive rule

- ****Default to `$derived`: if the local value is a pure transform of the prop (formatting, filtering, computing), reach for `$derived`**** — it is always correct, never goes stale, and costs nothing when nobody reads it.
- ****Reach for `$state(prop)` only when the child must own its own edits**** to the value (form inputs, draft text, in-progress edits). Accept that you then own the synchronization back to the prop.
- ****Avoid the legacy `let x = prop`**** it neither tracks the prop nor participates in reactivity — a silent source of stale-UI bugs.

Editor.svelte

```
<script>
  let { initialText = '' } = $props();

  // Editable local copy – initialized once from the prop.
  let draft = $state(initialText);

  // Read-only mirror – recomputes whenever initialText
  changes.
  let preview = $derived(draft.toUpperCase());
</script>

<input bind:value={draft} />
<p>Preview: {preview}</p>
```

- ① **Summary.** Use `$state(prop.value)` for an editable local copy initialized once from a prop, and `$derived(prop.value)` for a read-only computed mirror that re-syncs automatically. `$state(prop)` alone does not track later prop changes — pair it with a `$effect` only when you need both editability and continuous sync.

